COMPTE RENDU DE TRAVAUX PRATIQUES

Programmation pour cible embarquée

Commande d’une LED par accès direct aux registres

Étudiant	**Yousri Elouej**, Anis Boubakri, Mohamed Amine Bouguila
Classe	LISI 3
Microcontrôleur	STM32F407VG — ARM Cortex-M4
Environnement étudié	Keil µVision / MDK-ARM
Langage / méthode	C — accès direct aux registres CMSIS

Objectif du TP. Configurer PA5 comme sortie du STM32F407VG, inverser périodiquement son état logique, puis utiliser Keil µVision pour compiler le programme, programmer la cible et observer son exécution au débogueur.

Les photographies de la section 6 documentent les deux états observés de la LED. La réussite de la compilation et du téléchargement Flash doit être vérifiée dans Keil µVision à partir des messages affichés lors de l’exécution de la procédure.

1. Objectif et principe de fonctionnement

Ce TP vise à commander une sortie numérique du microcontrôleur STM32F407VG sans utiliser les fonctions HAL de commande GPIO. Le programme active l'horloge du port GPIOA, configure la broche PA5 en sortie, puis inverse son état dans une boucle infinie.

Le principe est le suivant :

1. Activer l'horloge du port GPIOA dans le registre RCC.
2. Configurer PA5 en sortie générale à l'aide du registre GPIOA->MODER.
3. Inverser le bit 5 du registre GPIOA->ODR.
4. Introduire une attente logicielle avant de recommencer.

2. Matériel et environnement

• Carte équipée d'un STM32F407VG et interface de programmation/débogage compatible (par exemple ST-LINK, selon la carte).
• Ordinateur avec Keil µVision / MDK-ARM et la chaîne de compilation ARM installée.
• LED et résistance de limitation de courant, par exemple 330 Ω, si aucune LED n'est déjà raccordée à PA5.
• Câble et alimentation adaptés à la carte.

Le nom du microcontrôleur ne permet pas, à lui seul, de savoir si une LED embarquée est reliée à PA5. Le brochage et le schéma de la carte doivent être vérifiés avant l'essai.

2.1. Montage électrique sur breadboard

Le montage expérimental comprend la carte STM32F407, une breadboard, une LED, une résistance de limitation de courant et des fils de connexion. La carte est reliée à l'ordinateur par USB pour l'alimentation et la programmation.

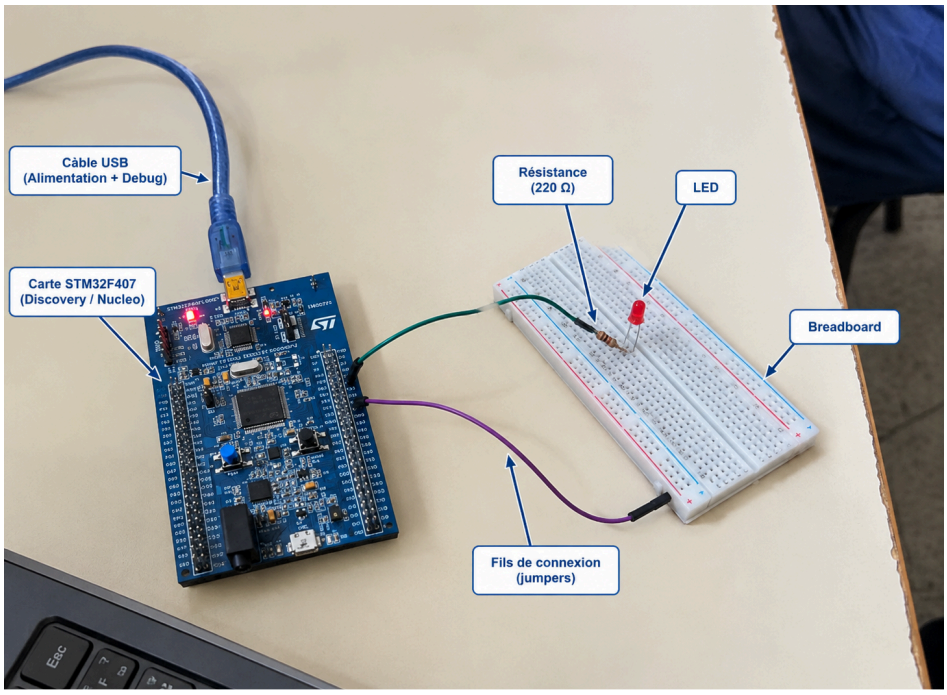


Illustration 1 — Vue explicative du montage électrique sur breadboard.

La résistance doit être montée en série avec la LED pour limiter le courant. Les photographies du montage réellement réalisé et les deux états observés de la LED sont présentés dans la section 6. Le câblage doit correspondre à la broche configurée dans le programme.

3. Interface Keil µVision et projet utilisé

La capture ci-dessous montre l'environnement de travail utilise pour le projet « tpp1 ». L'arborescence de gauche contient notamment main.c et startup_stm32f407xx.s. L'editeur affiche ici le fichier assembleur de demarrage.

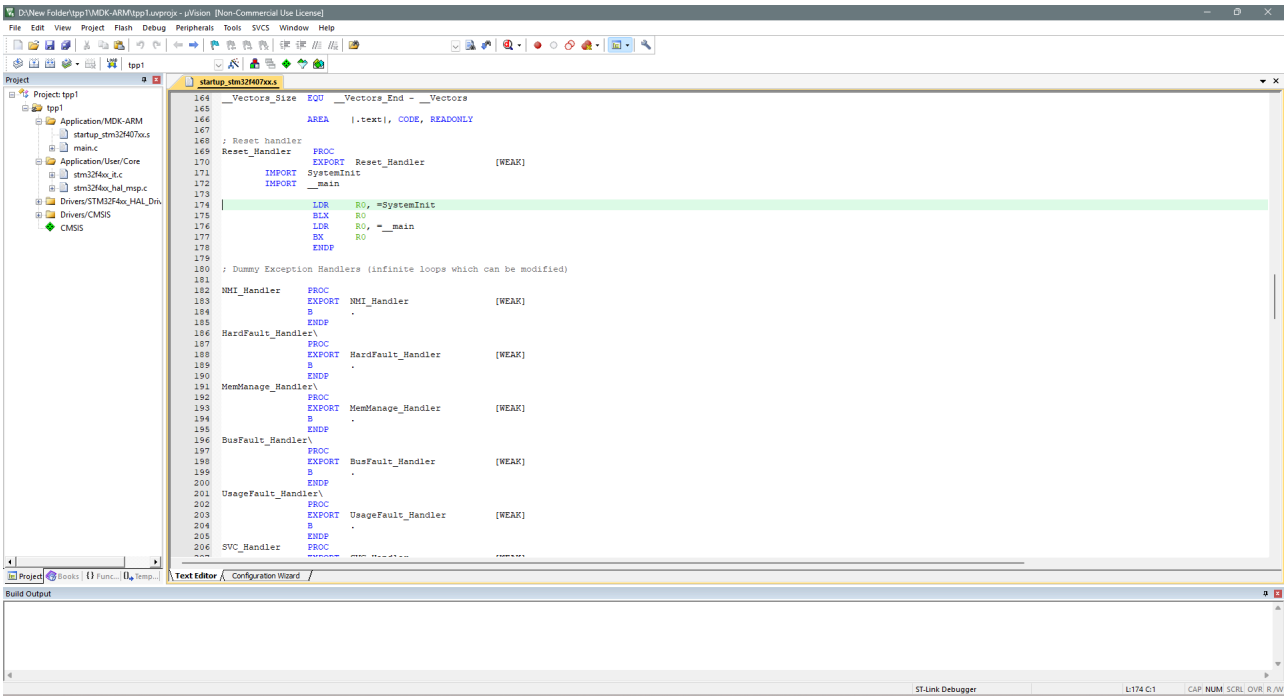


Figure 2 — Interface Keil µVision : projet tpp1, fichier de démarrage ouvert.

Pour consulter le programme applicatif, double-cliquer sur main.c dans l?arborescence du projet. Le fichier startup initialise l?exécution du microcontrôleur et transfère ensuite le contrôle vers le programme principal.

4. Programme C et emplacement des instructions

Le code ci-dessous représente la logique d'accès direct aux registres décrite dans ce compte rendu. Dans le projet existant, conserver la fonction main() déjà présente et intégrer les instructions dans sa structure ; ne pas créer une seconde fonction main(). Vérifier que les définitions CMSIS du STM32F4 sont incluses par le projet.

```
int main(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    GPIOA->MODER &= ~(3U << (5 * 2));
    GPIOA->MODER |= (1U << (5 * 2));

    while (1)
    {
        GPIOA->ODR ^= (1U << 5);
        for (volatile int i = 0; i < 10000; i++);
    }
}
```

4.1. Explication ligne par ligne

int main(void)

Déclare la fonction principale du programme. L'exécution du code applicatif arrive ici après le démarrage du microcontrôleur. *void* indique qu'aucun argument n'est attendu.

RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

Active l'horloge du port GPIOA. L'opérateur **|=** met à 1 le bit GPIOAEN sans modifier les autres bits du registre.

GPIOA->MODER &= ~(3U << (5 * 2));

Chaque broche GPIO dispose de deux bits de mode. Pour PA5, les bits concernés sont MODER[11:10]. L'expression crée un masque sur ces deux bits, l'inverse avec **~**, puis les efface grâce à **&=**.

GPIOA->MODER |= (1U << (5 * 2));

Écrit la valeur binaire **01** dans MODER[11:10], ce qui configure PA5 en sortie générale.

while (1)

Boucle infinie, car la condition 1 est toujours vraie. Le microcontrôleur répète les instructions à l'intérieur de la boucle.

GPIOA->ODR ^= (1U << 5);

Construit un masque dont seul le bit 5 vaut 1, puis inverse ce bit avec l'opérateur XOR **^=**. Le niveau de PA5 alterne entre 0 et 1 ; les autres bits ne sont pas ciblés par le masque.

for (volatile int i = 0; i < 10000; i++);

Boucle d'attente logicielle : i augmente de 0 à 9999. Le point-virgule final indique un corps vide. **volatile** limite certaines optimisations sur les accès à i, mais ne crée pas un délai précis ni calibré.

4.2. Registres et opérateurs utilisés

Élément	Rôle dans le programme
RCC->AHB1ENR	Active l'horloge du port GPIOA.
GPIOA->MODER	Configure le mode de PA5 ; 01 signifie sortie générale.
GPIOA->ODR	Contient les niveaux de sortie ; le bit 5 commande PA5.
|=	Met à 1 les bits sélectionnés sans modifier les autres.
&= ~masque	Efface les bits sélectionnés.
^= masque	Inverse les bits sélectionnés.

5. Procédure dans Keil µVision : Build, Flash et Debug

5.1. Compiler le projet — Build

1. Ouvrir **main.c** et enregistrer les modifications avec Ctrl+S.
2. Choisir **Project** → **Build Target** ou appuyer sur **F7**.
3. Lire les messages dans **Build Output** en bas de la fenêtre.
4. En cas d'erreur, corriger la première erreur pertinente puis relancer Build. Une compilation réussie doit être confirmée par le résumé affiché dans µVision.

Compiler le code et vérifier s'il y a des erreurs comme le montre la figure suivante.

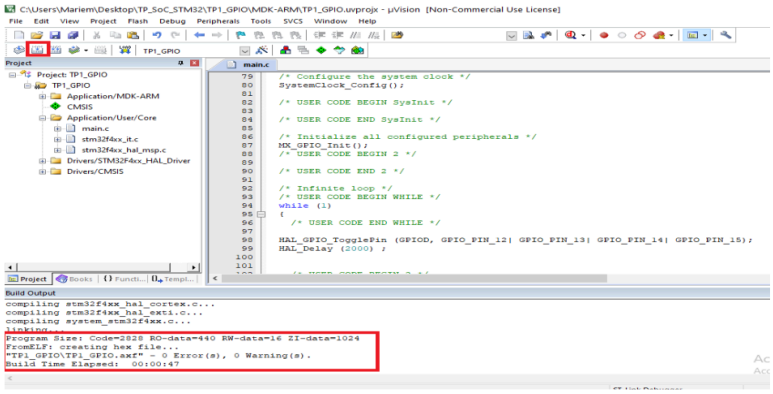


Figure 3 — Exemple de fenêtre Build Output provenant du document de référence ; elle illustre la procédure, et non le résultat de compilation du projet tpp1.

5.2. Programmer la mémoire Flash — Load / Flash

Après le Build, vérifier que la carte est alimentée et connectée. Dans les options de la cible, le composant sélectionné, l'interface de débogage et l'algorithme Flash doivent correspondre au matériel réellement utilisé. Lancer la commande de téléchargement Flash configurée dans µVision et vérifier le message de fin. Selon la configuration, le chargement peut aussi être déclenché automatiquement lors du démarrage de Debug.

4- Programmer la carte et tester le fonctionnement

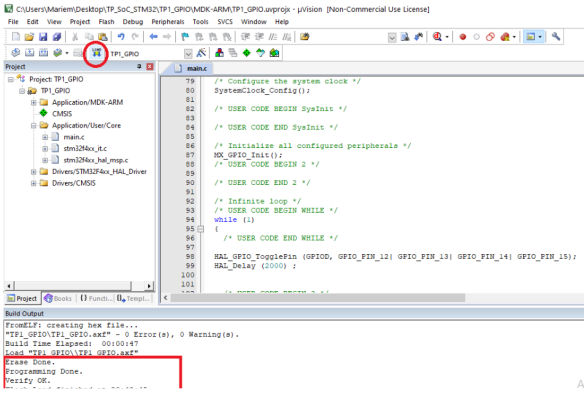


Figure 4 — Exemple de téléchargement Flash provenant du document de référence ; le résultat affiché ne constitue pas une preuve de programmation du projet tpp1.

5.3. Démarrer Debug et exécuter le programme

1. Ouvrir **Debug** → **Start/Stop Debug Session** (raccourci courant : Ctrl+F5).
2. Si la configuration est correcte, le débogueur ouvre la session sur la cible.
3. Utiliser **F5 (Run)** pour exécuter ou reprendre le programme.
4. Utiliser **F11 (Step)** pour avancer pas à pas et **F10 (Step Over)** pour passer par-dessus un appel de fonction.
5. Quitter la session avec Debug → Start/Stop Debug Session.

5- Lancer une session de debogage et executer le code sur la carte en appuyant sur « run »

comme le montre la figure suivante

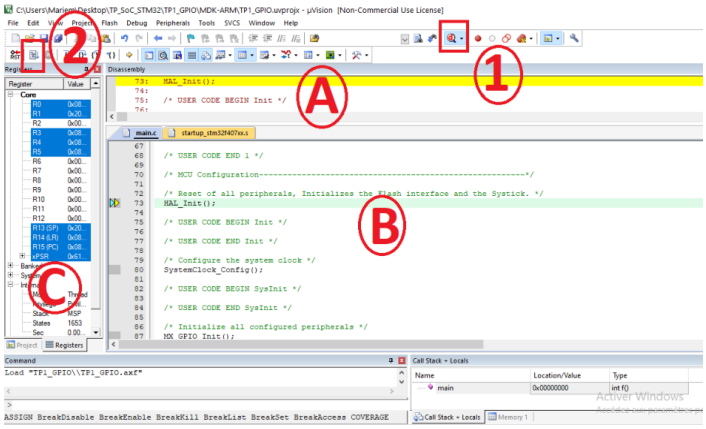


Figure 5 — Exemple de fenêtre de débogage issu du document de référence.

Zone A — Désassemblage : représentation en assembleur des instructions exécutées.
Zone B — Code source C : affiche le code C et, si les informations de débogage sont disponibles, met en évidence la ligne courante.

6. Résultats expérimentaux

Les deux photographies ci-dessous documentent le montage réellement réalisé avec la carte STM32F407, la breadboard et la LED externe. Elles montrent deux états observés pendant la manipulation : LED allumée et LED éteinte.

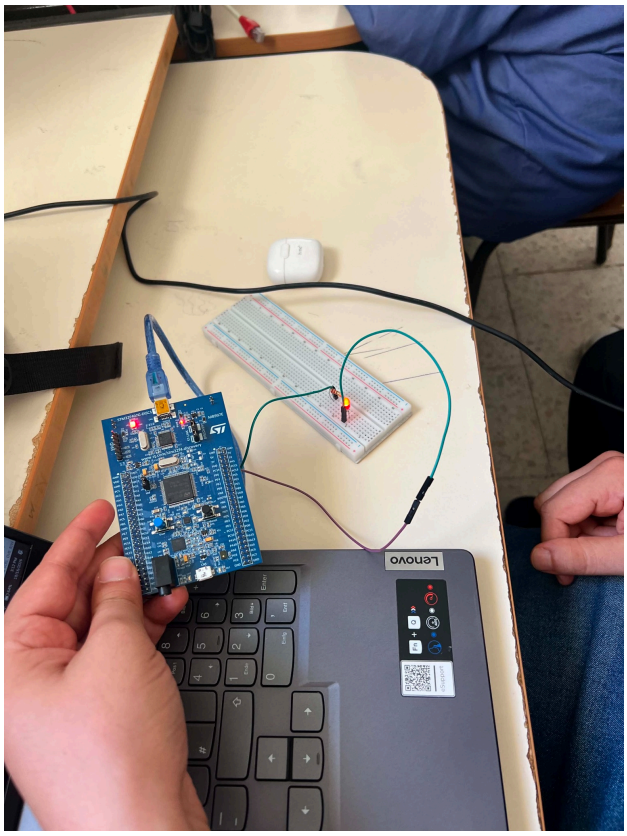


Figure 6 — Résultat observé : LED allumée.

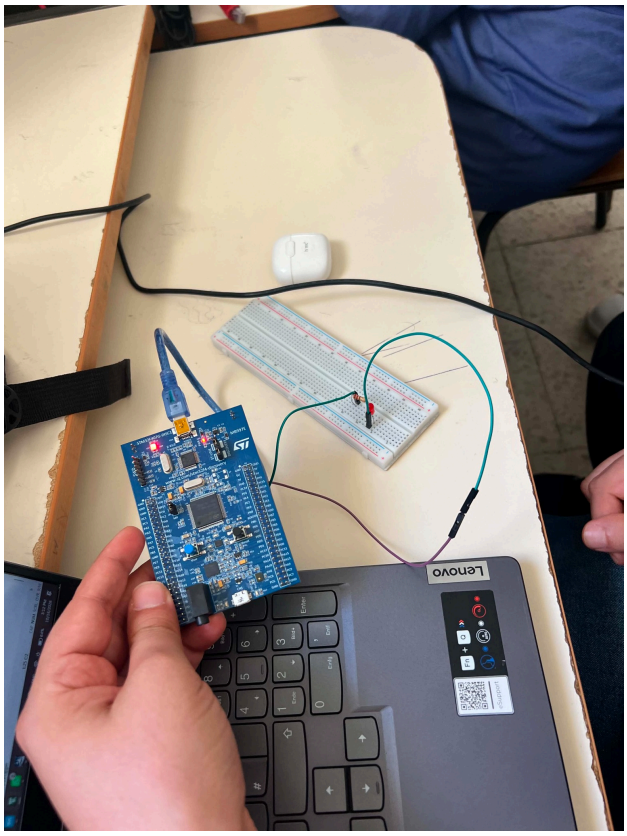


Figure 7 — Résultat observé : LED éteinte.

Interprétation : les deux états visuels de la LED sont compatibles avec une commande de sortie numérique. La broche exacte pilotée et la durée de commutation doivent être confirmées à partir du programme exécuté et du câblage réel.

7. Résultats attendus et validation

Contrôle	Résultat attendu	État / preuve
Compilation	Aucune erreur dans Build Output.	À vérifier dans le projet tpp1.
Programmation Flash	Téléchargement terminé sans erreur.	À confirmer sur la cible connectée.
Sortie PA5	Alternance du niveau logique 0/1.	À observer ou mesurer.
LED	Clignotement si la LED est raccordée correctement à PA5.	Essai matériel non confirmé dans les éléments fournis.

La boucle de 10 000 itérations peut être trop rapide pour produire un clignotement nettement visible. Pour tester visuellement,

8. Problèmes possibles et solutions

Symptôme	Cause possible	Vérification / solution
Erreur RCC / GPIOA	En-tête CMSIS absent ou cible mal configurée.	Vérifier le composant, le pack STM32F4 et les inclusions.
Build échoue	Erreur de syntaxe ou fichier non intégré au projet.	Lire Build Output et corriger les premières erreurs.
ST-LINK non détecté	Câble, pilote, alimentation ou configuration Debug.	Vérifier USB, pilote et interface SWD dans les options de cible.
LED ne clignote pas	LED non raccordée à PA5, polarité incorrecte ou délai trop court.	Vérifier le schéma, la résistance, la masse et le délai.

9. Limites et conclusion

La temporisation par boucle logicielle n’est pas précise : sa durée dépend de la fréquence du processeur, des options d’optimisation et de la configuration du projet. Le mot-clé volatile ne transforme pas la boucle en temporisateur calibré. Pour une temporisation stable, il est préférable d’utiliser un timer matériel ou une temporisation correctement configurée basée sur SysTick.

Ce TP permet de comprendre les bases de la commande d’une sortie numérique par accès direct aux registres du STM32F407VG. RCC active l’horloge du GPIOA, MODER configure PA5 en sortie, et ODR combiné au XOR inverse périodiquement son état. Dans Keil µVision, la démarche consiste à compiler avec Build (F7), programmer la Flash avec l’outil configuré, puis démarrer Debug pour exécuter ou inspecter le code.